

2-week HOME EXAM

PG3401 C Programming

Permitted aids: All

Duration: 14 days

Grading scale/evaluation type: National grading scale A - F

Date: April 23rd - May 7th, 2026

The exam text is in English only, you can hand in your exam in either English or Norwegian (Swedish and Danish is also accepted). Programming language used should though be the language C-89, other languages than C will not earn points.

The exam text has 11 pages. There are a total of 6 tasks in the exam set.

There is a 2-week deadline for this home exam. The period may overlap with other exams you have, so it is important that you use the time efficiently and plan your time well, so that you do not sit right before the deadline and have to submit several exams at the same time. Please note that the exam **MUST** be submitted within the deadline set in Wiseflow, and the assignment can only be submitted via WISEFLOW. It will not be possible to submit the assignment after the deadline – this means that you should submit well in advance so that you can contact the exam office or support if you have technical problems.

It is emphasized that the student must answer the exam independently and individually; cooperation between students and plagiarism is not allowed, the answer should represent the student's own work. This also means that it is not allowed to use code- or text-generating tools based on artificial intelligence. Be aware that attempts at "bluffing" on practical assignments in the form of screenshots found on the internet, screenshots received by other students or edited screenshots give negative points on the total score on the exam (which may result in the final grade being reduced by one or more grades), and in serious cases it will result in the filing of a cheating/plagiarism case. You should only document what you have achieved yourself, not try to make it look like you have achieved something you have not actually achieved. The exam must be solved on Linux.

Note that the tasks are made with increasing difficulty, and especially the last three tasks are more difficult than the first tasks. It is therefore encouraged to finish the first assignments (completely) so that the student does not spend all the time doing the most difficult tasks first.

Format of submission

This is a practical programming exam (apart from task 1), so the focus should be on explaining how you have proceeded to solve the tasks, justifying your choices and presenting any assumptions you have made in your solution.

If you are unable to solve a task, it is better if you submit what you have done (even if it does not work), and explain how you have tried to solve problems you faced and what you did not achieve – than not answer the task at all. It is expected that everything works unless otherwise described in the PDF file, code that the student knows does not work should also be commented as such in the code. If you know that the program is crashing, not compiling or not working as intended, it is important to explain this along with what steps you have taken to try to solve the problem.

The submitted source code must be in 1 ZIP file, the name of the file must be PG3401_V26_[candidate number].zip. This file should have the following structure:

```
\ logdata \ ewa_log.txt
\ logdata \ ewa_data.bin
\ task2_[candidate number] \ makefile
\ task2_[candidate number] \ [...]
\ task3_[candidate number] \ makefile
\ task3_[candidate number] \ [...]
[...]
```

In Wiseflow, you should upload a text answer with the name "PG3401_V26_[candidate number].pdf", and the ZIP file should be uploaded as an attachment to the text answer.

Be sure all files are included in the ZIP file (this includes all executable files, all makefiles, as well as the contents of the logdata folder). Each task-folder should have a makefile file, and no changes, third-party components or parameters should be required - the assessor will in shell on Debian Linux 10 go into the folder and type "make" and this should build the program with GCC. On all programming tasks, it is an **absolute requirement** to use the makefile file used during this course – as showed on **Lecture 4, slide 65 with the title «Use this makefile»**, you can make minor changes to the makefile file if necessary to solve the task, however, tasks that use a different makefile, regardless of whether it is auto-generated or created manually, will result in no points being awarded on the task. The referred lecture is available at the following URL: http://www.eastwill.no/pg3401/pg3401_lecture04.pdf

The text answer should contain an answer to assignment 1 (write it to the point, does not need a long thesis – however it should still be an answer of academic quality and style as expected of all work at college level). After the plain text answer, there should be 1 page of justification / documentation for each assignment, each of the documentation segments should be on a new page to make the text answer clear to the assessor. The answer must be in PDF format and have the correct file extension to be opened on both a Linux and a Windows machine (.pdf). Answers in other formats will not be read. (The text answer in the PDF file can be written on Mac or Windows, or any other operating system the student may be using, installing word processor on Linux is not required for this course.)

Required preparation

Before you start solving the programming tasks you need to download the tool EWA; this is new for the exams in 2025 and 2026 and has been covered during lecture 9.

You can either download the application from the domain 'eastwill.no' or download it from Canvas where it is found on the PG3401 coursepage under EKSAMEN (EXAM).

The student MUST use Debian 10 64 bit compiled for Intel (for students using VmWare Workstation on Windows or other Intel based systems) or compiled for ARM (for students using VmWare Fusion on Mac using M1, M2, M3 or M4 systems). No other versions of the application will be provided during the exam as only these 2 systems are supported for this course.

To download from 'eastwill.no' follow these steps, the file must be downloaded, then unpacked, and execute privileges must be set manually. Note that it MUST be run from a location on your system where you have write-access or it will fail (no help will be given during the exam, understanding how to do this is required knowledge).

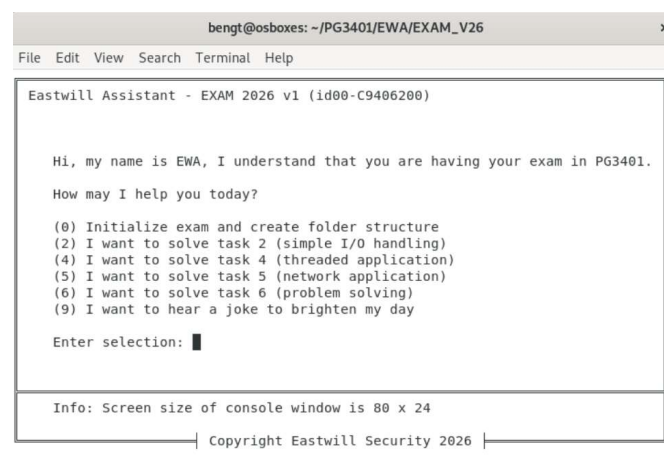
ZIP file is also found under EKSAMEN on Canvas, it can be copied in manually (from a USB drive for instance) if you don't have internet access in your VM. Access to the internet is not required for any task – but a functioning network adapter that can communicate on localhost (127.0.0.1) is required for Tasks 5 and 6.

VmWare Workstation (Intel based) or x64 Fusion (Mac with Intel CPU):

```
wget http://www.eastwill.no/pg3401/exam26/ewaexamv1_exam26_intel64.zip
unzip ewaexamv1_exam26_intel64.zip
chmod +x ewa_exam_v26_i64
./ewa_exam_v26_i64
```

VmWare Fusion (ARM based):

```
wget http://www.eastwill.no/pg3401/exam26/ewaexamv1_exam26_arm64.zip
unzip ewaexamv1_exam26_arm64.zip
chmod +x ewa_exam_v26_a64
./ewa_exam_v26_a64
```



Select option 0 to initialize the exam, when prompted enter your candidate number on this exam. This will create the folder structure you will use on this exam which should look like this:

```
bengt@osboxes:~/PG3401/EWA/EXAM_V26$ cd pg3401_exam26/
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26$ cd logdata
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/logdata$ ls
ewa_data.bin  ewa_log.txt
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/logdata$ cd ..
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26$ cd task2_1042/
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_1042$ ls
include  obj
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_1042$
```

Task 1. Theory (5 %)

- Explain what the C programming language can be used for.
- Who is Richard Stallman and what is he known for in the field of Information Technology?
- Explain what the sudo command is used for on a Linux-based system.

Task 2. File management and functions (15 %)

For this task you must first do the “Required preparation” as described above. In the EWA tool make sure you have chosen 0 to create and initialize the folder structure, and have supplied your candidate number for this exam.

In the EWA main interface select 2. This will create 8 source files that you will use for this task as well as a test file. The resulting output should look something like this:

```
Eastwill Assistant - EXAM 2026 v1 (id00-C9407647)

Hi, task 2 is a good start on this exam, it is a simple I/O task,
to solve it you need to make a makefile to link several files together,
the application then needs to read a file, call the new functions on the
data and write a new file. If you want I can help to test the result.

I have created a testfile 'pg3401_exam26/task2_1042/pgexam26_test.txt'
Use this file and the exam text to create your client application.

Ready to test the result? Press 1 to test, 0 to do it later: █

If you are now ready to deliver your exam you can take the entire folder
'pg3401_exam26', create a ZIP file with all the files in that folder, and
upload to WiseFlow together with your PDF report.

If you want to solve more tasks you can start me again, you can run
each task as many times as you want until you are satisfied with your
result and ready to deliver (just remember the deadline in Wiseflow :-))

Thank you for using EWA, and have a nice day.

bengt@osboxes:~/PG3401/EWA/EXAM_V26$ cd pg3401_exam26/
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26$ cd task2_1042/
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_1042$ ls
include  task2_abun.c  task2_fib.c  task2_perf.c
obj      task2_cube.c  task2_kvad.c  task2_prim.c
pgexam26_test.txt  task2_def.c  task2_odd.c
bengt@osboxes:~/PG3401/EWA/EXAM_V26/pg3401_exam26/task2_1042$ █
```

```
/pg3401_exam26/task2_[candidate number]/task2_fib.c
/pg3401_exam26/task2_[candidate number]/task2_prim.c
/pg3401_exam26/task2_[candidate number]/task2_kvad.c
/pg3401_exam26/task2_[candidate number]/task2_cube.c
/pg3401_exam26/task2_[candidate number]/task2_perf.c
```

```
/pg3401_exam26/task2_[candidate number]/task2_abun.c
/pg3401_exam26/task2_[candidate number]/task2_def.c
/pg3401_exam26/task2_[candidate number]/task2_odd.c
/pg3401_exam26/task2_[candidate number]/pgexam26_test.txt
```

The test file `pgexam26_test.txt` contains 11 integers where each number is on a line (there are line breaks between each number).

You should write a program that reads the numbers in the text file, for each of the numbers in the text file you should call the functions in the above source files to find if the numbers are a) a number in the Fibonacci series, b) a prime number, c) a square number, d) a cube number, e) a perfect number, f) an abundant number, g) a deficient number, or h) an odd number.

The program must create an output file where each number is printed BINARY, along with metadata about the number according to the results from the functions listed above. Each number in the output file should be described with the following struct:

```
struct TASK2_NUMBERS_METADATA {
    int iIndex; // Sequence number, where first number = 1
    int iNumber; // The number itself, as read from file
    int bIsFibonacci;
    int bIsPrimeNumber;
    int bIsSquareNumber;
    int bIsCubeNumber;
    int bIsPerfectNumber;
    int bIsAbundantNumber;
    int bIsDeficientNumber;
    int bIsOddNumber;
}
```

You need to create a makefile file (use the makefile file taken from lecture 4, slide 65 – with the heading "Use this makefile") that links together the source files, and one or more header files for the program to compile correctly. Make any changes required to the makefile file and the source files to get the program to build and run correctly.

The program should create an output file where this structure is written as BINARY, this file must be named `pgexam26_output.bin` and be placed in the same folder as the input file the EWA tool has created for you.

Required documentation:

You must make a screenshot of YOUR program when it is running, you must also make a screenshot of EWA after you have pressed 2 and it has created the files you will use in your program. You also need to run the EWA tool again after you have completed your program and then ask EWA to verify your output file by pressing 1 – and make a screenshot after EWA has verified it. The screenshots need to be in the ZIP file and be named `task2_scrn1.png`, `task2_scrn2.png` and `task2_scrn3.png` (and should be placed in the same folder as your makefile) AND it must also be included

in the proper location in your PDF file.

The output file pgexam26_output.bin must also be part of the submitted ZIP file.

Task 3. List handling (20 %)

You will be creating a simple data structure to handle exam submissions and grading. The list should be a double-linked list of exam submissions, each item (struct) in the list should contain pointers to both the previous item and the next item. The item should also contain a text string that is CANDIDATEID (for example, 10042), fields for POINTS on each of the tasks, fields for JUSTIFICATION on each of the tasks, and a final SCORE that is automatically calculated from the sum of points on each task.

You will create functions that perform the following operations on the list:

- The code must include comments that explain what the code does, and be structured and organized to improve readability and maintainability
- Add an exam candidate manually
- Automatically add a list of exam candidates from a file
- Complete an assessment of one task by setting score and justification
- Print out a list of all candidates with their score and corresponding grade, the grade needs to be calculated according to a formula you code into your application
- Print out a list of all candidates that have accomplished a supplied grade (for instance the user can ask for all candidates with the grade A, and your application will print a list of candidates), according to the same formula you used above
- Print out a list of all candidates that have not been graded yet (by checking for entries in the list where not all of the tasks have received a score and justification
- Print out the grade as well as the score and justification on each task for a given candidate

You are going to create a main function that receives instructions from the user based on input from the keyboard, the main must be able to call all functions above (for example, in some form of menu) and ask the user for data necessary to call the mentioned functions. The use of the application should be optimized logically to grade a set of exam papers, use this exam as the model for the system (which

includes number of tasks, and maximum points per task). Main should clean up all data before it returns (an option in the menu must be to exit).

Required documentation:

You must make one screenshot when your application starts (showing the menu screen), and one of the application when it is running one of the “tasks”. The screenshots need to be in the ZIP file and be named task3_scrn1.png and task3_scrn2.png (and should be placed in the same folder as your makefile) AND it must also be included in the proper location in your PDF file.

Task 4. Threads (20 %)

In practical programming, it is often efficient to put time-consuming operations in worker-threads, examples of which are file operations, network operations, mathematical operations, and communication with external devices. In this assignment, you will simulate such operations with a smaller data set – in order to save you time during the exam, a ready-made application has already been created for you to change.

Application consists of 3 threads, the main thread (running the main function) and 2 worker threads. The main thread creates the worker threads (in this application, the main thread has no other function and only creates the two threads that do the work).

Both worker threads read numbers from a text file and perform a mathematical test to determine if the number is a prime number or not. When prime numbers are found the worker threads add those numbers to a linked list. When both threads have completed the main thread reads the linked list and prints out the prime numbers on the screen.

For this task you must first do the “Required preparation” as described above. In the EWA make sure you have chosen 0 to create and initialize the folder structure, and have supplied your candidate number for this exam.

In the EWA main interface select 4. This will create 4 files that you will use for this task; 3 source files with the code described above, and a txt file to be used for testing.

```
Eastwill Assistant - EXAM 2026 v1 (id01-C9408447)

Hi. Task 4? Great. Task 4 on the exam is a multithreading task,
to solve it you need to make a makefile to build the application.
I have written task4_threads.c for you, and you need to change it
according to the task description in the exam paper.
I might have an error in there, it didnt compile, you need to fix that...

I have created a testfile 'pg3401_exam26/task4_1042/task4_primes.txt'
Change task2_threads.c and run your application.

Ready to test the result and see part II?
Press 1 to test, 0 to do it later: █
```

Copyright Eastwill Security 2026

```
/pg3401_exam26/task4_[candidate number]/task4_threads.c  
/pg3401_exam26/task4_[candidate number]/task4_prime.c  
/pg3401_exam26/task4_[candidate number]/include/task4_prime.h  
/pg3401_exam26/task4_[candidate number]/task4_primes.txt
```

The source files are a solution to the application as was described above. task4_threads.c contains the threading code and the main function (all though it is missing some key features). The test file task4_primes.txt contains a list of large numbers.

PART I

You should not change task4_prime.c (it is suboptimally implemented, otherwise it would be too quick to test the threading code – do not optimize the algorithm). The following changes must be done in task4_threads.c, do NOT create new files, and do NOT move the file to other folders (the EWA tool will read the file when you are ready, and expects it to be at the same location/file as it created it):

- You must create a makefile file (as detailed above under “Format of submission”) to build the program, make any changes required to the makefile file and source file to make the program build and run correctly
- The program uses global variables, change this to that all variables are local variables in the main function and submitted as parameter to the two threads
- Main() has hardcoded the name of the file to be read, change the program so it takes the file name as a parameter on the command line and passes this on to the call to fopen
- The program is missing synchronization code, add mutexes to correctly protect the shared resources
- Add code for explicit initialization of mutex (with the *_init functions)
- Add comments to your code to document what the code does, your comments must clearly explain where the mutex use has been added, what problems would arise without mutex in place (in other words in the original provided code) and how your use of mutexes solves that

PART II

- If you are able to complete the first part of the task, the tool EWA will prompt you for some more changes you will make to your final solution.
- You need to (at least) partially succeed on Part I to be shown Part II in the EWA tool, this is required because Part II builds upon Part I of this task
- Once you have succeeded sufficiently the EWA tool will give you the next part of this task – your progress is shown in the tool

Required documentation:

You must take a screenshot of the EWA tool after you have completed part I (if you were able to do so). Your final ZIP file must include all files, including any output files from Part II (if you were able to complete Part II).

Task 5. Network (20 %)

For this task you must first do the “Required preparation” as described above. In the EWA make sure you have chosen 0 to create and initialize the folder structure, and have supplied your candidate number for this exam.

In the EWA main interface select 5. This will create a header file and a sample file:

```
/pg3401_exam26/task5_[candidate number]/ include / ewpdef.h
```

In this header file you will find some define values and data structures. You are going to create a server application (and the EWA tool has implemented a client that will be used for testing your server). In order to run this you need to open 2 terminal windows; one running your application, and one running EWA.

The server application should be executed with port number and a user-specified ID on the command line, such as "task5 -port 25 -id Smtptest". When started, the application should open the specified port for LISTEN, for this task it is enough to bind to loopback on 127.0.0.1, and communication must use TCP.

You should simulate traffic between client and server (the traffic is “inspired by SMTP”), points will be given for implementations that are flexible and robust, with error handling that can correctly handle data received from the client (especially if it is wrong – or malicious).

Communicating with EWA involves the following steps:

- EWA will establish a TCP connection with your application (the server), EWA will attempt to connect to 127.0.0.1 and the port you provide to EWA
- Your server must send a “server accept” message to the client
- After receiving a HELO from the client, the server must reply with a code 250 message that includes the clients IP address, and gives a greeting message
- After receiving the MAIL FROM from the client, the server must reply with 250
- After receiving the RCPT TO from the client, the server must reply with 250
- After receiving the DATA command from the client, the server must reply with a 354 message if the name supplied can be used as a valid filename (this deviates from the SMTP protocol, see the protocol defined in the header file), or an error message if it cannot be used as a valid filename
- Any data received from the client should be stored in this file, data might come as one or more data streams, and ending with a CRLF CRLF . CRLF (a single dot on a separate line), when this “end of file” marker is received the file must be saved and closed by the server
- The client can then send either a QUIT command, or send a new file by sending a new DATA command
- When QUIT is received the server application should send a 221 message,

and can then shut down the server application

The task should not be solved using Curl or other third-party libraries and should not be based on launching other applications in the operating system - only the use of Sockets as we have learned in lecture 10 on Network will give points on the task.

Required documentation:

You must take a screenshot of the program when it is running, the screenshot must be attached to the answer in an image file named task5_screenshot.png (in the same folder as your makefile) AND must also be inserted in a suitable place in the text answer (PDF file). The screenshot must include both your running application, as well as the EWA tool after you have received a file (if you were able to do so). Any files received by the EWA tool must also be included in your exam answer, the files should have the filename as was sent as part of the DATA command and be stored in the same folder as your makefile (pg3401_exam26/task5_[candidate number]).

Task 6. Problem solving (20 %)

This task is intended as a more difficult task than the previous, and it will combine different techniques and functions with the need for some problem solving on your part. This task might use code that is part of the other tasks, so solving this task might be more difficult if you have not already completed the previous tasks.

For this task you must first do the "Required preparation" as described above. In the EWA make sure you have chosen 0 to create and initialize the folder structure, and have supplied your candidate number for this exam.

You will create an application that works as a network client, the EWA tool will bind to a port on the loopback adapter 127.0.0.1, and will listen on a random port, the port is displayed in the EWA tool when the port is ready to send data.

Your application must accept the IP address and port number of the server as parameters from the terminal, for instance:

```
./task6 -server 127.0.0.1 -port 42420
```

When you connect to the EWA tool, first select option 6 in the main interface, and it will then send you a file. The protocol used is a custom implementation best described as "TCP over TCP"-protocol. You need to use the protocol as defined in:

```
/pg3401_exam26/task6_[candidate number]/include / ewpdef.h
```

You then need to create the code that uses this protocol correctly so the resulting code is able to receive the file sent. Your code need to handle out-of-sequence packets, check crc code, and send ACK or NACK responses as required. (Since NACK is not defined for TCP we use an existing value for NACK, see the header file.)

Note that some problemsolving and debugging needs to be expected to successfully implement the communication with EWA, the EWA tool will log errors if unexpected data is received which should serve as "hints" to what you need to change to comply.

The file received is in binary format, and contains a BMP image representing a butterfly. Finally, your code must save the file to disk, it should be saved in the same folder as your makefile.

Required documentation:

You must take a screenshot of the program when it is running, the screenshot must be attached to the answer in an image file named task6_scrnshot.png (in the same folder as your makefile) AND must also be inserted in a suitable place in the text answer (PDF file). The screenshot should show both your application and EWA. The file received must also be included in the exam submission as task6_received.bmp.

+

End of task set